Insper

ENGENHARIA

CYBERSEGURANÇA EM NUVEM

Atividade em grupo – Gerenciamento de recursos em nuvem

Eduardo Machado de Oliveira

Gustavo Eizo Takahashi

Henrique Leite dos Santos

Lucas Kenji Kamikawa

Sidney Barbosa de Oliveira

Prof. Dr. Rodolfo Avelino

São Paulo

03/11/2026

Sumário

Contexto do ambiente (MaisPlace) 3
Questão 1 – Acesso administrativo e identidade sobre os recursos 4
1.1 Tabela de recursos, acessos e privilégios 4
1.2 Comprometimento do Guacamole e do token da Cloudflare 5
1.3 Roles e usuários IAM e o princípio do menor privilégio 5
Questão 2 – Monitoramento do uso dos recursos 6
2.1 O que se ganha ao centralizar os registros no Wazuh 6
2.2 Sinais monitorados e camada de cada um 6
2.3 A pergunta que cada ferramenta responde 7
Ligação com um incidente do caso MaisPlace 8

Contexto do ambiente (MaisPlace)

O ambiente analisado é o marketplace **MaisPlace** (maisplace.net.br, WordPress + WooCommerce), construído nos Roteiros 1 a 3. A conclusão central do grupo é que **centralizar o acesso no Guacamole reduziu a superfície de ataque, mas concentrou o risco**: o Guacamole e o servidor web, que guarda o token da Cloudflare, tornaram-se os recursos que mais precisam de controle.

O que foi efetivamente configurado nos roteiros:

- **Servidor web:** EC2 t3.micro Ubuntu (North Virginia) com Apache, PHP, WordPress e certificado via certbot. O Security Group libera 80/443 somente para a Managed Prefix List com os IPs da Cloudflare e SSH somente a partir do Security Group do Guacamole. O acesso direto pelo IP público resulta em timeout (Roteiro 2, Tarefa 3).
- **Banco de dados:** EC2 com MySQL em sub-rede privada, sem IP público, com `bind-address` no IP privado. O Security Group aceita 3306 e 22 apenas do Security Group do servidor web. Há um usuário MySQL próprio para a base do WordPress.
- **Guacamole:** EC2 Ubuntu dedicada, com Docker Compose (guacamole, guacd e postgres) atrás de Nginx. O Security Group aceita apenas HTTPS da Cloudflare; o Nginx possui allowlist dos IPs dos administradores; o TOTP é obrigatório; a porta 8080 está presa ao loopback e o UFW está ativo. A conexão SSH administrativa aponta para o IP privado do servidor web.
- **Cloudflare:** DNS, proxy e WAF com cinco regras: bloqueio geográfico (fora do Brasil), exploração de plugins, SQL injection, rate limiting em `/wp-admin` e bloqueio de `/wp-json/wp/v2/users`.
- **Resposta automatizada:** o **Fail2ban, no servidor web**, detecta 5 falhas de login em `/wp-login.php` em 10 minutos e chama a API da Cloudflare, que cria uma IP Access Rule de bloqueio por 1 hora. O token tem apenas permissão de editar regras de firewall da zona, só aceita chamadas do IP do servidor e fica em `cloudflare.env` com `chmod 600`.
- **Monitoramento:** Wazuh com os servidores conectados como agentes; CloudWatch com alarmes de CPU (acima de 70% por 5 minutos, com aviso via SNS), verificação do sistema (status check), consumo anormal de rede e uso de disco. O Zabbix foi considerado conforme previsto no enunciado da atividade.

Os pontos marcados como **"Recomendação"** nas seções seguintes ainda não estão implementados no ambiente e representam as melhorias propostas pelo grupo.

Questão 1 – Acesso administrativo e identidade sobre os recursos

1.1 Tabela de recursos, acessos e privilégios

A rede ficou bem fechada (nada administrativo exposto e tudo atrás da Cloudflare), mas os privilégios **dentro** dos recursos ainda são mais amplos do que as tarefas exigem. Esses pontos aparecem na tabela como "Excesso".

Recurso	Quem / o que acessa	Privilégio	Concessão e revisão	Se comprometido
Servidor web	**Pessoas:** administradores via Guacamole (SSH no IP privado); administradores do WordPress pelo `/wp-admin`. **Sistemas:** visitantes, somente via Cloudflare (80/443 restritos à Prefix List); Fail2ban (com o token da Cloudflare); agente Wazuh.	Admin SSH: administrar (sudo). Admin WordPress: administrar o site e instalar plugins, o que equivale a executar código. Visitantes: leitura e compra. Fail2ban e agente Wazuh executam como root. **Excesso:** WordPress sem MFA (apontado no Roteiro 1); usuário SSH único com sudo.	**Hoje:** SSH apenas pelo Security Group do Guacamole e por conexão cadastrada nele; contas do WordPress criadas manualmente. **Recomendação:** um usuário Linux e uma conta WordPress por pessoa; MFA no WordPress; revisão mensal de contas, `authorized_keys`, plugins e regras do Security Group; remoção imediata quando alguém deixa o grupo.	Defacement, skimming de cartão no checkout e roubo de sessões. O atacante lê o `wp-config.php` (senha do banco) e o `cloudflare.env` (token). Como o Security Group do banco aceita 3306 **e 22** vindos do web, o servidor web é a ponte para todo o restante.
Banco de dados (sub-rede privada)	**Sistemas:** WordPress, com usuário MySQL próprio, a partir do servidor web; agente Wazuh. **Pessoas:** administradores por SSH; como o Security Group só aceita SSH vindo do web, o caminho é Guacamole → web → banco.	WordPress: privilégios na própria base (no Roteiro 1 houve concessão de privilégios, provavelmente `ALL`). Admin: root do MySQL e sudo no Linux. **Excesso:** `ALL` inclui DROP/ALTER, usados apenas em atualizações; SSH no banco liberado para todo o Security Group do web.	**Hoje:** usuário criado manualmente; isolamento por sub-rede privada, sem IP público, `bind-address` no IP privado e Security Group referenciando o do web. **Recomendação:** liberar a porta 22 somente para o Security Group do Guacamole (com conexão própria) e retirá-la do web; usuário do WordPress apenas com SELECT/ INSERT/UPDATE/DELETE no dia a dia; revisão trimestral de usuários e grants; rotação da senha do `wp-config.php`.	Vazamento de todos os dados de clientes, pedidos e vendedores do marketplace (LGPD), alteração de preços e pedidos, apagamento de dados ou ransomware. É o recurso de maior impacto para o negócio.
Guacamole (jump server)	**Pessoas:** os cinco integrantes do grupo pelo portal web (HTTPS via Cloudflare, IP na allowlist do Nginx, senha + TOTP). **Admin do Guacamole:** quem cria usuários e conexões. **Sistemas:** guacd e Postgres em contêineres (o Postgres guarda as credenciais das conexões).	Usuário comum: apenas as conexões atribuídas. Admin do Guacamole: cria usuários e conexões, ou seja, administra indiretamente **todo** o ambiente. **Excesso:** se a conexão SSH salva usa um usuário com sudo, quem a abre já é root no web; um usuário administrativo compartilhado elimina a rastreabilidade.	**Hoje:** TOTP obrigatório, allowlist de IP no Nginx, apenas HTTPS da Cloudflare no Security Group, UFW e porta 8080 no loopback. **Recomendação:** usuário nominal por integrante, permissões por grupo e poucos administradores; gravação de sessões; revisão mensal de usuários, conexões e histórico; atualizar a allowlist quando o IP de alguém muda ou alguém sai.	**Ponto único de falha:** o atacante herda as conexões salvas e chega ao web (e de lá ao banco) com credenciais legítimas, sem acionar o Fail2ban nem regras de força bruta. Também pode impedir o acesso dos administradores.

Recurso	Quem / o que acessa	Privilégio	Concessão e revisão	Se comprometido
Instância Wazuh / Zabbix	**Pessoas:** integrantes pelo dashboard web e SSH via Guacamole. **Sistemas:** agentes nos servidores web, banco e Guacamole; integração CloudWatch → Wazuh.	Analista: leitura de alertas. Admin: regras, agentes, decoders e active response, que executa comandos como root nos agentes. Integração AWS: deve ter apenas leitura dos grupos de log. **Excesso:** dashboard com um único usuário admin (padrão da instalação).	**Hoje:** Wazuh instalado com os servidores conectados. **Recomendação:** dashboard acessível apenas via Guacamole/ allowlist; usuários nominais com perfis de leitura e de administração; chave de enrollment para agentes; credencial AWS por IAM role somente leitura; revisão trimestral de usuários, regras e agentes.	O atacante "cega" a defesa: desativa regras, apaga alertas e esconde a própria atividade. Com active response, executa comandos em todos os servidores. O Zabbix entrega o mapa completo do ambiente.
Conta e token da API Cloudflare	**Pessoas:** dono da conta e quem administra DNS/WAF. **Sistemas:** action do Fail2ban no servidor web, com o token em `cloudflare.env`.	Token: apenas editar regras de firewall da zona maisplace.net.br e somente a partir do IP do servidor (menor privilégio bem aplicado). Conta: administrar DNS, proxy, WAF e certificados. **Excesso residual:** "editar firewall" também permite **criar regras de allow** e apagar bloqueios, não apenas bloquear.	**Hoje:** token criado com escopo mínimo e restrição de IP; `chmod 600`; o script lê o token de variável de ambiente, sem hard-code. **Recomendação:** data de expiração e rotação a cada 90 dias; MFA e membros nominais na conta; revisão do audit log da Cloudflare; alerta para qualquer chamada que não seja de bloqueio.	A restrição de IP não protege se o próprio servidor web for invadido, pois o atacante chama a API a partir do IP autorizado. Ele pode liberar o próprio IP (contornando o WAF), apagar bloqueios ou bloquear clientes legítimos. Com a conta Cloudflare: troca de DNS para um site falso (phishing) e exposição do IP real.

1.2 Comprometimento do Guacamole e do token da Cloudflare

- **Guacamole:** o maior risco não é a força bruta, já dificultada por TOTP, allowlist e Cloudflare, e sim o roubo de sessão ou uma falha no próprio Guacamole/Postgres que exponha as credenciais salvas. Com isso, o atacante teria acesso administrativo legítimo a todos os servidores. **Medidas:** manter as imagens Docker atualizadas; não salvar senha ou chave com sudo na conexão (cada um digita a sua) ou usar um usuário sem sudo e elevar apenas quando necessário; gravar as sessões; enviar os logs de login do Guacamole ao Wazuh com alertas para horário ou origem incomuns; manter um acesso emergencial pelo AWS Systems Manager Session Manager, controlado por IAM role, para não depender apenas dele.
- **Token da Cloudflare:** o escopo mínimo já impede trocar o DNS ou desligar o proxy, mas permite manipular as regras de firewall. **Medidas:** como o token reside no servidor web, sua segurança depende da segurança do web; o ideal é mover a chamada à API para outra máquina (por exemplo, o active response do Wazuh) ou para uma função Lambda com o token no Secrets Manager; definir expiração; monitorar o audit log e revogar o token ao menor sinal de abuso.

1.3 Roles e usuários IAM e o princípio do menor privilégio

- Como respondido no Roteiro 1, **usuários IAM** são indicados para identidades fixas (pessoas que acessam o console) e devem ter MFA; **roles** fornecem credenciais temporárias e são o caminho para serviços. No nosso ambiente, o CloudWatch Agent (necessário para o alarme de disco) e a integração CloudWatch → Wazuh devem usar **IAM roles** com permissão mínima, sem chaves de acesso gravadas em arquivo.

- O token da Cloudflare equivale a um "usuário" com credencial fixa: por isso precisou de escopo mínimo e restrição de IP, e ainda falta definir sua expiração. Se fosse um recurso da AWS, seria uma role.
- O **menor privilégio** aparece em todas as linhas da tabela: o token só edita o firewall, o Security Group do banco só aceita o web e o Guacamole só abre as conexões atribuídas. Onde ele ainda falha (WordPress sem MFA, usuário MySQL com `ALL`, SSH no banco a partir do web), o comprometimento de um recurso se espalha para os demais.
- O Guacamole faz para as pessoas o papel que as roles fazem para os serviços: é nele que o acesso administrativo é concedido, limitado e revisado, pois a AWS não enxerga quem está por trás de uma sessão SSH aberta por ele.

Questão 2 – Monitoramento do uso dos recursos

Concentrar os logs do CloudWatch no Wazuh transforma registros isolados em **correlação**: um mesmo ataque deixa rastros na infraestrutura e nos servidores, e somente com a visão conjunta é possível reconhecê-lo e reagir a tempo.

2.1 O que se ganha ao centralizar os registros no Wazuh

- **Correlação entre camadas:** o alarme de rede anômala do CloudWatch, sozinho, pode ser apenas uma promoção no marketplace. Somado a um login no `/wp-admin` vindo de IP novo e à alteração de um arquivo de plugin (Wazuh), torna-se um indício claro de invasão com exfiltração.
- **Linha do tempo única** para investigação: um só lugar e uma só busca, sem alternar entre o console da AWS e o dashboard do Wazuh. É o que o grupo respondeu no Roteiro 1 sobre SIEM: coleta, análise e detecção, e não apenas armazenamento.
- **Regras padronizadas**, inclusive mapeadas ao MITRE ATT&CK, em vez de limiares isolados em cada console.
- **Visão além do Fail2ban:** hoje a resposta automática enxerga apenas o `/wp-login.php` de um servidor. Com tudo no Wazuh, o mesmo raciocínio pode valer para eventos da nuvem e de outros hosts, por exemplo acionando o bloqueio na Cloudflare a partir de um alerta do Wazuh.
- **Retenção e integridade:** uma cópia dos registros fica fora do local atacado; se o atacante apagar logs locais ou do CloudWatch, a cópia permanece no Wazuh, o que também apoia auditoria e LGPD.
- **Menos fadiga de alertas:** um painel priorizado em vez de e-mails do SNS, alertas do Zabbix e do Wazuh chegando separadamente.

Ressalva: o Wazuh passa a ser um recurso crítico (ver Questão 1), e o CloudWatch e o Zabbix continuam necessários para capacidade e disponibilidade, que não são o foco dele.

2.2 Sinais monitorados e camada de cada um

Sinal	Camada	Alerta no nosso ambiente
Uso de CPU das instâncias	CloudWatch	Acima de 70% por 5 min, com aviso por e-mail via SNS (Roteiro 3)
Verificação do sistema (status check da EC2)	CloudWatch	Falha no status check da instância ou do sistema (Roteiro 3)
Tráfego de rede (NetworkIn / NetworkOut)	CloudWatch	Acima do limite definido pelo grupo em 1 min (Roteiro 3)

Sinal	Camada	Alerta no nosso ambiente
Uso de disco	CloudWatch (via CloudWatch Agent)	Disco acima do limite definido (Roteiro 3)
Disponibilidade do site e do banco (HTTP, porta 3306, ICMP)	Zabbix	Site sem resposta em checagens seguidas; MySQL recusando conexões
Memória, serviços e processos (Apache, MySQL, contêineres do Guacamole)	Zabbix	Serviço parado; memória esgotada na t3.micro
Tempo de resposta do site	Zabbix	Latência acima do aceitável para o checkout
Logins SSH, uso de sudo e logins no Guacamole	Wazuh	Login fora do horário, de origem nova ou com muitas falhas
Ataques web nos logs do Apache (SQLi, varredura de plugins, força bruta)	Wazuh (e Fail2ban para o `/wp-login.php`)	Padrão de SQL injection; 5 falhas de login em 10 min → bloqueio via API da Cloudflare
Integridade de arquivos (FIM)	Wazuh	Alteração em `wp-config.php`, plugins, temas, `/etc/passwd`, `sudoers`, `cloudflare.env`
Logs da AWS encaminhados (CloudWatch Logs / CloudTrail)	Wazuh (correlacionado)	Alteração de Security Group, Prefix List ou policy IAM
Chamadas à API da Cloudflare	Wazuh (log do Fail2ban) + audit log da Cloudflare	Chamada que não seja bloqueio de IP, ou criação de regra de allow

2.3 A pergunta que cada ferramenta responde

- **CloudWatch:** "Nossa infraestrutura na nuvem está saudável e bem dimensionada, ou está sendo usada de forma anormal?"
- **Zabbix:** "A loja está no ar e respondendo bem para o cliente neste momento?"
- **Wazuh:** "Alguém está tentando, ou conseguiu, fazer algo indevido no nosso ambiente, e já reagimos?"

Ligação com um incidente do caso MaisPlace

Com o gerenciamento e o monitoramento atuais, um ataque de **força bruta na administração** seria bloqueado em segundos. Já um **vazamento de dados de clientes por plugin/WooCommerce desatualizado** seria dificultado e detectado mais cedo, mas não evitado. Ambos estão entre os riscos que o grupo mapeou para o MaisPlace no Roteiro 1, ao lado de SQL injection, phishing e movimentação lateral.

Incidente: vazamento de dados de clientes a partir do painel administrativo

Cenário: um atacante obtém acesso de administrador ao WordPress (por senha vazada ou por falha em plugin desatualizado), instala um plugin malicioso e extrai a base de clientes e pedidos.

- **Teria evitado:**
  - Força bruta no `/wp-login.php`: o Fail2ban bloqueia o IP na Cloudflare na 6ª tentativa, e o rate limiting do WAF em `/wp-admin` interrompe o ataque antes; o bloqueio geográfico elimina boa parte dos robôs estrangeiros.
  - Exploração de caminhos conhecidos de plugins e payloads de SQL injection: bloqueados pelas regras do WAF.
  - Acesso direto ao banco ou ao SSH: impossível a partir da Internet (banco sem IP público, SSH somente via Guacamole com TOTP, 80/443 somente da Cloudflare).
- **Não teria evitado:** login com **senha correta vazada** (phishing ou reuso), pois o WordPress não possui MFA, e uma vulnerabilidade de plugin desconhecida pelo WAF. A partir do painel, o atacante instala código e lê o `wp-config.php` com a senha do banco.
- **Teria detectado mais cedo:**
  - Wazuh (FIM): novo plugin ou arquivo PHP alterado em `wp-content`, e leitura ou alteração do `wp-config.php`.
  - Wazuh: login de administrador vindo de IP ou horário incomum nos logs do Apache.
  - CloudWatch: alarme de tráfego de rede anormal no servidor web durante a exfiltração, e CPU alta caso o atacante instale um minerador.
  - Correlacionando esses sinais no Wazuh, o incidente seria confirmado em minutos, em vez de ser descoberto por reclamação de clientes.
- **O que falta para fechar a lacuna:** MFA no WordPress, atualização ou remoção de plugins, usuário MySQL sem `ALL`, SSH do banco liberado somente para o Guacamole e alerta automático no Wazuh para instalação de plugins.

Conclusão: o menor privilégio reduz o raio de impacto e o monitoramento correlacionado reduz o tempo de detecção; nenhum dos dois resolve sozinho. No MaisPlace a rede já está bem fechada, e o próximo passo é aplicar o mesmo rigor às identidades dentro dos recursos (WordPress, MySQL e usuários do Guacamole).